

Pi GUI 调用 Paper2Agent 技术实践

Python 环境 技能安装与可核验的执行流程

Stella 2026 年 9 月 19 日

在 Pi GUI 的输入框里写下 `/skill:paper2agent`，看见 Agent 开始工作，只是这条流程的起点。

真正影响任务能否完成的，往往是后面几个问题：Skill 的脚本有没有随目录一起安装，Pi 调用的是哪个 Python，终端能找到的工具为什么 GUI 找不到，以及“严格验证通过”究竟验证了什么。

本文以一次 Windows 实测为基础，讲清 Pi GUI 调用 Paper2Agent 的技术路径。目标是生成有来源、有审查记录、可以交付给 Agent 读取的论文阅读包。科学训练环境单独说明，不把它当成安装 Paper2Agent 的前提。

一套 Pi 核心完成论文工具调用

界面负责交互 Pi 编排工具 Python 执行转换



图 1 Pi GUI 到本地转换脚本的调用架构示意。文件需通过被识别的路径打开；并非落盘后一定自动进入预览列表。

测试使用 Pi 0.85.1，Paper2Agent 固定到提交 8c2d059165ef8cdcb70dbea76655b9c2b55b38e6。下文 PowerShell 命令使用 `C:\paper-lab` 作为可替换示例目录；历史测试实际运行在独立测试目录中，不代表本文又在一台干净电脑上重装验证了一遍。

1 一套 Pi 核心的调用架构

这里的 Pi GUI 指 Stella Pi Workbench。它负责会话、模型选择、工具结果展示和文件预览；Paper2Agent 的转换脚本运行在本机 Python 进程里，不在浏览器页面里。

项目代码中，主进程直接解析 @earendil-works/pi-coding-agent/rpc-entry，由 PiRpcRuntime 启动并通信。接入这个 Skill 不需要修改 Pi 核心，也不需要再部署一个 Team 服务。

Skill 是执行说明和配套文件，Python 是脚本运行环境，模型负责理解任务与调用工具。把这三层分清，排错会容易很多。

2 Python 与命令执行环境

本次用到的环境不是 “一套 Python 装全部依赖”：

三个用途分别管理依赖

论文转换

Python 3.12 + uv
PyMuPDF 与 PDF 依赖
读取脚本内依赖声明
不需要 TensorFlow

科学实验

Python 3.10.11
DeepChem 2.8.0
TensorFlow 2.15.1
RDKit 与 scikit-learn

Paper2Any 组件

Python 3.12.13
独立虚拟环境
大纲与页面生成依赖
不覆盖科学环境

这些是本次实际版本 不是所有论文的统一依赖清单

图 2 本次实际环境的隔离方式。Paper2Skill 不依赖科学训练环境，Paper2Any 的 Python 环境也单独维护。

用途	本次实际环境	何时需要
Pi 原生会话	Pi 0.85.1、已配置的模型	GUI 中执行任务
本地命令	Windows shell；实测使用 Pi 的 bash 工具	运行转换脚本
论文转换	uv 0.11.7、Python 3.12	Paper2Skill 阅读包
MoleculeNet 训练	Python 3.10.11、独立虚拟环境	需要重跑科学实验时
Paper2Any Python 服务	Python 3.12.13、另一独立环境	另外测试大纲等组件时

Paper2Agent 此版本的 `paper_bundle.py` 声明最低 Python 3.11，并通过脚本内元数据声明依赖：

参数与依赖

```
pymupdf==1.28.2
pymupdf4llm==1.28.2
pypdf==6.18.1
pillow==12.2.0
```

这些是该固定版本的实际声明，不是建议永远锁在这些版本。[转换脚本源码](#)

使用 `uv run --python 3.12` 执行时，uv 会根据脚本声明准备依赖环境。无需为了阅读一篇 PDF，就向系统 Python 安装 TensorFlow、PyTorch 或一整套 Office 服务。[uv 脚本运行文档](#)

Windows 还要留意 shell。Pi 的默认 bash 工具会寻找 Git Bash；“电脑有 PowerShell”不等于这个工具必然能用。安装 Git for Windows 通常可以提供 Git Bash。Pi 也提供可配置的 PowerShell 工具，但本次历史测试用的是 bash 路线。下文的配置命令明确写为 PowerShell，不应原样放进 Bash 执行。

Pi Windows 说明

先按 [uv 官方安装说明](#) 安装工具，再在 PowerShell 检查：

PowerShell

```
git --version
uv --version
uv python install 3.12
uv python find 3.12
```

如果刚修改过 PATH，需要重新启动 Pi GUI，让应用继承新的环境。仅在某个终端里设置的变量，也不会自动进入已经打开的 GUI 进程。

3 安装完整的 Paper2Agent Skill

先取得固定版本源码。以下目录不存在时才进行这次新建检出：

PowerShell

```
$PaperWorkspace = 'C:\paper-lab'
$Paper2AgentRepo = Join-Path $PaperWorkspace 'Paper2Agent'
New-Item -ItemType Directory -Path $PaperWorkspace -Force | Out-Null

git clone https://github.com/jmiao24/Paper2Agent.git $Paper2AgentRepo
if ($LASTEXITCODE -ne 0) { throw 'Paper2Agent 源码获取失败' }

git -C $Paper2AgentRepo checkout --detach 8c2d059165ef8cdcb70dbea76655b9c2b55b38e6
if ($LASTEXITCODE -ne 0) { throw '固定版本检出失败' }

git -C $Paper2AgentRepo rev-parse HEAD
```

要导入的目录是 Paper2Agent\skills\paper2agent，不是仓库根目录，也不是单独的 SKILL.md。其结构大致如下：

完整 Skill 目录

```
paper2agent/
├── SKILL.md
├── paper2skill/
│   ├── SKILL.md
│   ├── scripts/
│   └── references/
├── paper2mcp/
│   └── ...
└── paper2agent-paper/
    └── ...
```

根入口负责选择分支。只复制入口文件，Agent 后续就可能读不到 paper2skill/scripts/paper_bundle.py 和审查规范。

本次固定版本的 Windows 补丁

测试时发现, `paper2skill/scripts/reading_package.py` 中用于比对包内路径的代码, 采用了:

Python

```
rel = str(path.relative_to(root))
```

Windows 会产生反斜杠, 导致路径比较误报。测试副本的修正是:

Python

```
rel = path.relative_to(root).as_posix()
```

如果按本文固定提交重现, 应在导入前核对并应用这一处修改, 保留差异记录。它只改变校验所用的路径表示, 不删除校验项。使用新版本时先检查上游是否已修复, 不要盲目重复替换。

这一补丁只在测试副本生效, 未向上游提交。

GUI 中的安装路径

在 Stella Pi Workbench 中选择论文工作目录, 然后进入 “PI 原生工作台 → Skills 管理”, 点击 “添加 Skill 文件夹”, 选择上述完整目录。

安装范围有两种:

- 当前项目: 写入该项目的 `.pi/skills/`, 项目需要受信任。
- 所有项目: 写入当前 Pi Agent 数据目录下的 `skills/`; 默认通常为 `~/.pi/agent/skills/`。

Pi 还支持其他 Skill 搜索位置, 但本次安装说明只使用上述两种。若启用了 `PI_CODING_AGENT_DIR`, 全局安装目标应以这个实际目录为准, 不能只检查默认用户目录。[Pi Skill 搜索规则](#)

界面称之为 “安装并热加载”。从当前代码看, 实现方式是: 复制文件夹, 保留当前 Session 标识, 再重启 Pi 运行时加载资源。正在生成或压缩上下文时, 安装会被明确拒绝; 它不是对正在执行的任务无感替换脚本。

安装完成后, 应在 Skills 管理中看到 `paper2agent` 及其真实位置。导入本身不会替你安装 Git、Python 或科研依赖。

本次可视化测试为隔离用户环境，预先复制了完整 Skill 目录；之后验证了真实技能发现和调用。上述 G UI 文件夹导入与重载行为另外依据项目代码核对，不把它冒充本次从点击安装到运行的完整 E2E 记录。

4 在 GUI 中发起论文任务

选择工作项目，确认模型已配置，并先做一次真实连接测试。历史测试使用 DeepSeek V4.1 Flash，但这不是 Skill 规定的唯一模型。连接测试通过后，还要确认工具调用正常；若需在本轮审查 PDF 图片，执行环境也必须具备相应的视觉读取能力。

然后回到当前会话，输入 `/skill:paper2agent`。

历史测试中，技能候选正常出现，Tab 可以确认；补充任务要求后按 Enter，Pi 真正开始执行工具，技能长正文以可展开摘要显示，没有全部铺进聊天记录。尚未结束的任务也能在左侧列表中看到。

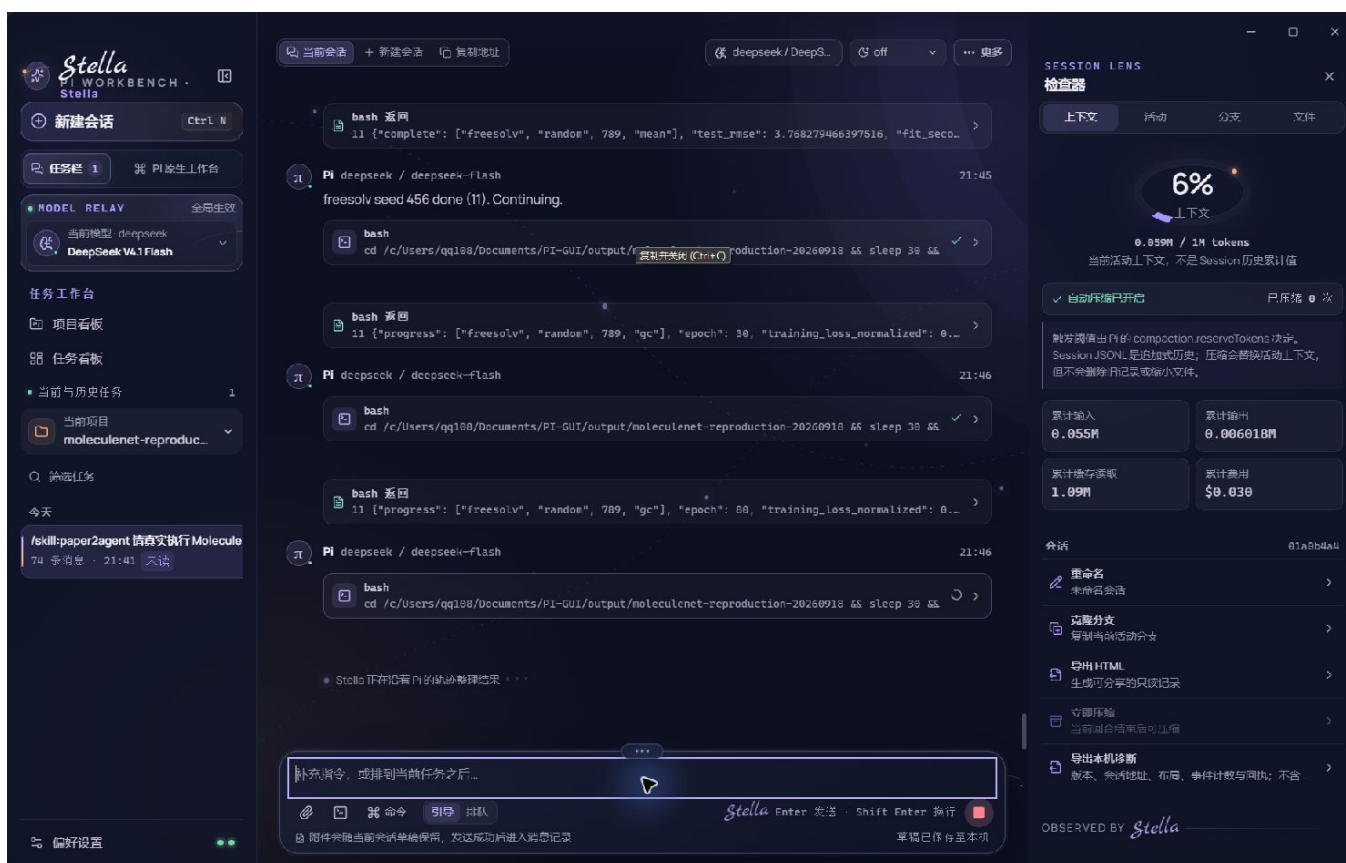


图 3 2026 年 9 月 18 日真实 Pi GUI 截图。左侧任务在运行中可见，中间持续显示工具回执，输入框和停止按钮保留。右侧数值仅是当时界面快照。

下面是一段可改路径后使用的提示词。它是根据测试整理的示例，不是当时会话的逐字转录：

可修改路径后使用的任务示例

```
/skill:paper2agent
```

请处理 C:\paper-lab\sources\molculenet-author-manuscript.pdf。

本次只走 Paper2Skill，不做 Paper2MCP，不启动子代理。

请使用 uv 和 Python 3.12，先报告实际解释器及脚本位置。

工作目录为 C:\paper-lab\review-molculenet；

最终阅读包为 C:\paper-lab\deliverables\molculenet-paper。

已有目录不要覆盖，继续已有任务前先核对来源。

保存来源快照，执行 prepare、extract、review-aid。

按照该 Skill 的审查规范核对全部页面、公式、图表、单位和脚注。

无法可靠转成文本的内容保留原图，并明确限制。

如果当前环境不能查看页面图片，请明确报告审查未完成，

不要把未查看的页面标记为 reviewed。

审查完成后 build --require-reviewed，并执行 verify --strict。

交付阅读包位置、verification.json、具体限制和来源哈希。

不要把“生成阅读包”描述为“论文实验复现完成”。

最后仅根据阅读包，回答一个关于数据集划分或评估指标的问题，

并指明对应章节或图表。

这里特意限定 Paper2Skill，因为完整 Paper2MCP 还有源码执行、工具包装、独立代理验证和 MCP 运行时交付等要求。本次没有满足或验证那一整套流程，不能因为两者共用根入口就混为一谈。

Paper2MCP 官方要求

5 转换命令与审查步骤

为了排错，也为了能独立复核 Agent 的行为，最好知道 GUI 背后实际执行什么。

Paper2Skill 的执行与审查路径



review-aid 不是已审查证明 退出码 0 仍可能是 reviewed_with_limitations

图 4 转换与审查流程示意。逐页审查不能由 review-aid 命令替代，严格验证通过也需要继续阅读具体状态与限制。

以下是按本次命令参数整理的 PowerShell 示例。先将 PDF 保存到给定路径，并从 Skills 管理复制“已安装副本”的位置；示例采用项目级安装：

PowerShell

```
$PaperWorkspace = 'C:\paper-lab'
$SkillRoot = Join-Path $PaperWorkspace '.pi\skills\paper2agent'
$BundleScript = Join-Path $SkillRoot 'paper2skill\scripts\paper_bundle.py'
$PaperPdf = Join-Path $PaperWorkspace 'sources\moleculenet-author-manuscript.pdf'
$ReviewDir = Join-Path $PaperWorkspace 'review-moleculenet'
$BundleDir = Join-Path $PaperWorkspace 'deliverables\moleculenet-paper'

if (-not (Test-Path -LiteralPath $BundleScript -PathType Leaf)) {
    throw '未找到已安装的 Paper2Skill 脚本，请核对 Skill 位置'
}
if (-not (Test-Path -LiteralPath $PaperPdf -PathType Leaf)) {
    throw '未找到论文 PDF'
}

uv run --python 3.12 $BundleScript --help
if ($LASTEXITCODE -ne 0) { throw '转换脚本无法启动' }
```

创建来源快照

PowerShell

```
uv run --python 3.12 $BundleScript prepare $PaperPdf --work $ReviewDir --name moleculenet-
paper --title 'MoleculeNet: a benchmark for molecular machine learning' --main $PaperPdf
if ($LASTEXITCODE -ne 0) { throw 'prepare 失败' }
```

检查生成的 `inventory.json` 和 `bundle.json`，确认输入、正文角色和来源哈希。prepare 要使用新的工作目录；已有目录应按实际状态恢复，而不是删除记录重来。

提取与审查

PowerShell

```
uv run --python 3.12 $BundleScript extract --work $ReviewDir
if ($LASTEXITCODE -ne 0) { throw 'extract 失败' }
```

```
uv run --python 3.12 $BundleScript review-aid --work $ReviewDir
if ($LASTEXITCODE -ne 0) { throw 'review-aid 失败' }
```

到这里，不能直接写“转换完成”。下一步要按照 Skill 的 `references/review-plan.md`，对照页面图像检查正文顺序、公式、跨页内容、表格和脚注，并记录具体审查意见。

review-aid 生成的是联络图和待处理清单，不会替人或 Agent 看过所有页面。对扫描 PDF，OCR 还可能需对应语言数据；即使没有文本提取错误，公式和上下标也仍需视觉核对。[上游审查计划说明](#)

历史测试使用了 reuse-review，因为同一份 PDF 已由主代理完成过逐页审查，且源文件 SHA-256 完全相同。首次处理一篇论文时没有这个前提，不能复制别人的 `reviewed: true` 来代替自己的审查。本次也没有新增独立审查代理。

构建与严格验证

PowerShell

```
uv run --python 3.12 $BundleScript build --work $ReviewDir --output $BundleDir --require-reviewed
if ($LASTEXITCODE -ne 0) { throw '阅读包构建未通过' }
```

```
uv run --python 3.12 $BundleScript verify --work $ReviewDir --strict
if ($LASTEXITCODE -ne 0) { throw '严格验证未通过，请检查具体诊断' }
```

```
Get-Content -LiteralPath (Join-Path $ReviewDir 'verification.json') -Raw
```

如果有未解决的诊断，修正应回到审查计划和来源证据，再重新构建。不要直接改最终阅读包来消除报错，那会破坏生成记录与哈希的一致性。

本次实际结果是：

验证结果

```
{
  "status": "reviewed_with_limitations",
  "mechanical_ok": true,
  "all_sources_agent_reviewed": true,
  "issues": []
}
```

--strict 在 reviewed 和 reviewed_with_limitations 两种状态下都会返回 0，所以应同时阅读状态与限制，不能只检查退出码。这个例子的限制包括复杂表格保留为图片，以及复用了同源的主代理审查记录。[Paper2Skill 构建与验证定义](#)

最后还要做一次实际检索：只给 Agent 阅读包，要求它回答一个与论文方法有关的问题并定位来源。文件完整与内容可用是两项不同的检查。

在这次 GUI 测试中，文件落盘并不保证自动进入右侧文件列表。助手消息中的绝对路径用行内代码表示时能够被识别，用粗体包裹同一路径时却未被识别。因此交付时还应实际点击文件入口，确认可以打开，而不是只检查磁盘上有文件。

6 独立的科学实验环境

Paper2Skill 负责整理论文材料。重跑 MoleculeNet 的模型训练，是另外一层工作。

本次科学环境采用 Python 3.10.11，核心依赖如下：

参数与依赖

```
deepchem==2.8.0
tensorflow==2.15.1
rdkit==2023.9.6
scikit-learn==1.4.2
numpy==1.26.4
pandas==2.2.2
pytest==8.3.5
```

实际运行使用 Windows CPU、4 线程，没有 GPU。这里的版本组合来自本次记录，不是 Paper2Agent 对所有论文的通用要求。

若已经取得本次实验代码和数据，可在实验目录中创建独立环境，并明确调用其解释器：

PowerShell

```
# 在包含 requirements.txt、benchmark.py 和数据的实验目录执行。
# .venv-science 应为本次新建环境，不能覆盖已有环境。
uv venv --python 3.10.11 .venv-science
if ($LASTEXITCODE -ne 0) { throw '科学环境创建失败' }

uv pip install --python .\.venv-science\Scripts\python.exe -r requirements.txt
if ($LASTEXITCODE -ne 0) { throw '科学依赖安装失败' }

.\.venv-science\Scripts\python.exe --version
.\.venv-science\Scripts\python.exe benchmark.py --output results-new
if ($LASTEXITCODE -ne 0) { throw '科学实验执行失败' }
```

benchmark.py 是本次测试的配套代码，不是安装 Paper2Agent 就会自动出现的官方命令。新批次必须用新输出目录，并重新核对对应的原始数据、分区、预测和指标；旧的审计回执不能证明新批次通过。

历史实验共执行 48 次训练或基线评估。FreeSolv 的单位错误、ESOL 的重复结构、随机划分与骨架划分的区别，都需要由科学代码和数据审计处理。更换 GUI 或模型，不会自动解决这些问题。

Paper2Any 的 Python 组件也使用独立环境。那次测试选择 Python 3.12.13，并不意味着该项目声明“只支持 Python 3.12”；隔离的目的，是避免它的依赖解析结果覆盖已经验证的科学环境。

7 可视化测试发现的问题

这次 Pi GUI 测试，技能补全、Enter 发送、运行中任务可见、工具调用回执，以及右侧 Markdown 文件切换都实际通过了检查。

但仍有几个与长任务直接相关的问题。

长会话刷新失败。 在约 350 个持久化事件后，测试版本出现 Electron contextBridge recursion depth exceeded。后续请求仍被 Pi 接收，工具流也继续，但完整界面状态刷新失败。这说明“后台任务在跑”和“界面状态正确”必须分别验证。这个数值是该次观察点，不是已证明的固定触发阈值。

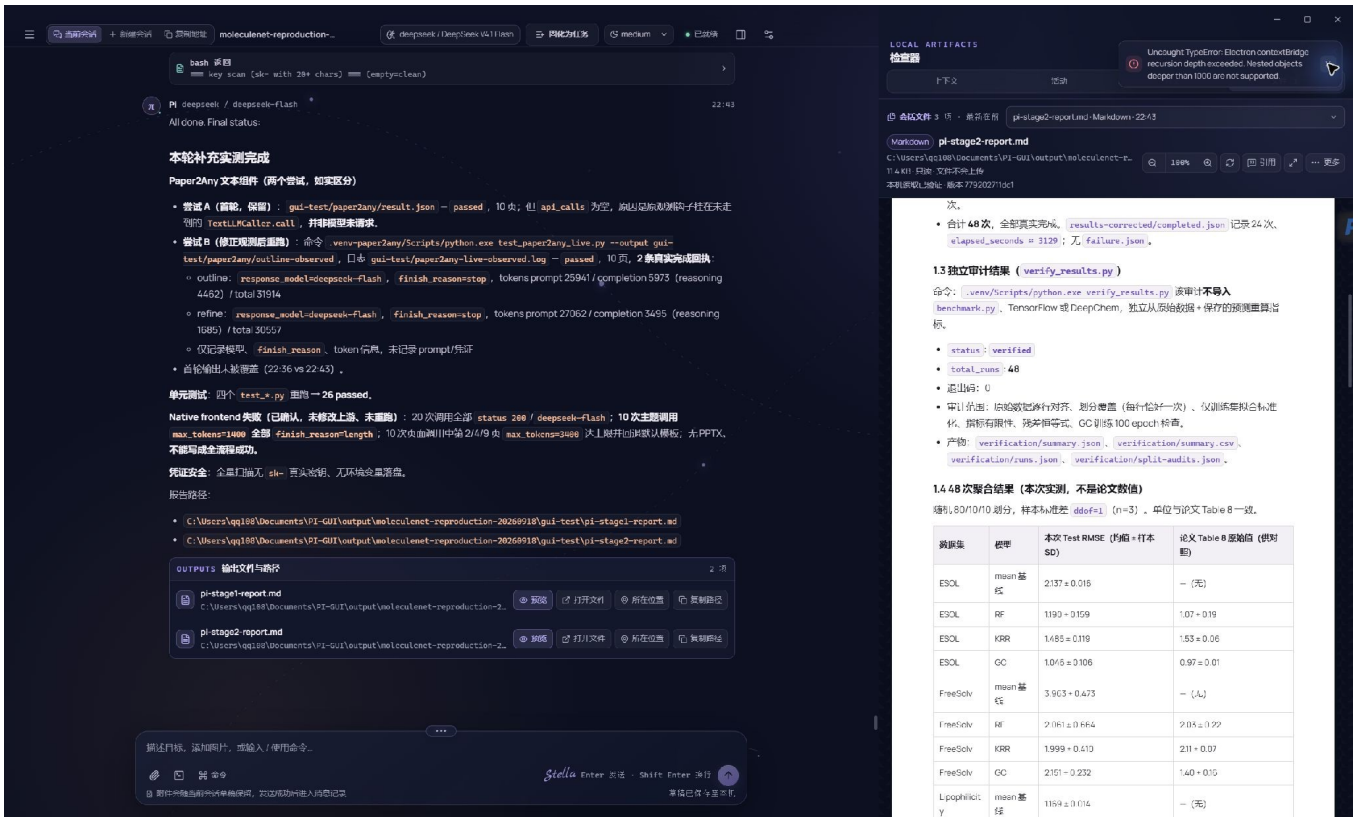


图 5 真实测试截图同时保留报告预览与右上角 contextBridge 错误。它说明界面可显示部分内容，但长会话全量状态刷新仍然失败。

停止等待不等于后台进程全部退出。 一次通过 nohup 启动的科学子进程，在 Pi 停止等待工具后仍继续运行。处理此类问题要核对具体进程、命令行和输出文件，不能看到会话停止就立即重复启动同一实验。

预览不等于源文件损坏。 PPTX 在窄右栏以 100% 显示时有裁切，手动缩到 70% 才能看到整页。PDF 页面视觉显示可用，但文本辅助提取出现提示。它们分别属于预览布局和辅助读取问题，不能仅凭界面表现认定原始文件有错。

这些是 2026 年 9 月 18 日测试快照中的观察，不是对所有后续版本的结论。本文没有重新进行全套 GUI 回归，也没有把这些问题标记成已修复。

排查时可以按现象缩小范围：

现象	优先核对
找不到 /skill:paper2agent	Skill 根目录、安装范围、项目是否受信任、实际 Agent 数据目录
能找到 Skill，却找不到脚本	是否只导入了 SKILL.md，是否选错文件夹层级

现象	优先核对
终端可运行，GUI 报找不到命令	GUI 继承的 PATH、解释器绝对路径、实际 shell
验证报告为未审查或有诊断	页面审查记录、来源哈希、具体错误，不应绕过校验
界面似乎卡住但日志继续增长	会话状态传输、后台进程和实际输出，避免重复执行

8 交付文件与验收依据

一次可检查的论文任务，至少应保留三类东西。

首先是阅读包本身，供后续 Agent 使用；其次是独立的审查目录，保存来源、审查记录和 verification.json；如果执行过实验，还应有解释器与依赖版本、运行参数、预测文件及核验结果。

Pi GUI 的作用，是把这些过程放到一个能观察、能追踪、能打开文件的工作台里。科学依赖仍留在独立环境，原始记录仍保留在文件系统，Pi 核心也不需要因为一个论文工具而再复制一套。

本次已证明的是：在明确环境和范围后，Pi GUI 可以真实调用 Paper2Agent 的 Paper2Skill 流程并产出经过验证的阅读包。完整论文复现、代码转 MCP，以及长任务交互稳定性，仍需各自的证据，不能由这一个结果代替。

测试与资料核查截至 2026 年 9 月 19 日。本文所列命令与参数依据固定版本源码及保存的运行记录整理。跨平台安装与完整 GUI 回归不在本次验证范围内。